FIT5197 Final Assessment

Student	SUN ZHEN
Student ID	36446874
Kaggle name	xiamiya
Regression public RMSE	4.92159; second selected 4.92162
Classification public macro F1	0.56078

FIT5197 2026 S1 Final Assessment

SPECIAL NOTE: Please refer to the assessment page (Clayton), or (Malaysia) for rules, general guidelines and marking rubrics of the assessment (the marking rubric for the kaggle competition part will be released near the deadline - around 5 working days - in the same page). Failure to comply with the provided information will result in a deduction of mark (e.g., late penalties) or breach of academic integrity.

PLEASE ENSURE YOU READ AND COMPLETE ALL REQUIRED INFORMATION BELOW BEFORE STARTING YOUR ASSIGNMENT, AS FAILURE TO DO SO WILL RESULT IN A MARK OF ZERO.

YOUR NAME: SUN ZHEN

STUDENT ID: 36446874

KAGGLE NAME (required for marking - see Part 1, Question 5 or Part 2): xiamiya

Important: Your Kaggle name must be identical (including capitalization, spacing, and special characters) across both Regression and Classification contests. Marks will be deducted if your Kaggle name is inconsistent or missing. No exceptions will be made!

You can find your Kaggle name by clicking on your profile icon at the top-right corner of the Kaggle website and go to your profile. For example, here is a screenshot of where to find it:

After that, the Kaggle name can be found similar to the one in the picture:

Please note that the name to be recorded is not "Dan Nguyen 11" but "dannguyen11". Marks will be deducted if you record the incorrect names.

Please also enter your details in this google form.

Part 1 Regression (50 Marks)

A few thousand people were questioned in a life and wellbeing survey to build a model to predict happiness of an individual. You need to build regression models to optimally predict the variable in the survey dataset called 'happiness' based on any, or all, of the other survey question responses.

You have been provided with two datasets, `regression_train.csv` and `regression_test.csv`. Using these datasets, you hope to build a model that can predict happiness level using the other variables. `regression_train.csv` comes with the ground-truth target label (i.e. happiness level) whereas `regression_test.csv` comes with independent variables (input information) only.

On the order of around 70 survey questions have been converted into predictor variables that can be used to predict happiness. We do not list all the predictor names here, but their names given in the data header can clearly be linked to the survey questions. e.g., the predictor variable 'iDontFeelParticularlyPleasedWithTheWayIAm' corresponds to the survey question 'I don't feel particularly pleased with the way I am.'

PLEASE NOTE THAT THE USE OF LIBRARIES ARE PROHIBITED IN THESE QUESTIONS UNLESS STATED OTHERWISE, ANSWERS USING LIBRARIES WILL RECEIVE 0 MARKS

Question 1 (NO LIBRARIES ALLOWED) (4 Mark)

Please load the `regression_train.csv` and fit a multiple linear regression model with 'happiness' being the target variable. According to the summary table, which predictors do you think are possibly associated with the target variable (use the significance level of 0.01), and which are the Top 5 strongest predictors? Please write an R script to automatically fetch and print this information.

NOTE: Manually doing the above tasks will result in 0 marks.

Code Cell 7

```
# Q1: Multiple linear regression using base R only.
train &lt;- read.csv("regression_train.csv")

lm.fit &lt;- lm(happiness ~ ., data = train)
model.summary &lt;- summary(lm.fit)
coef.table &lt;- as.data.frame(model.summary$coefficients)
coef.table$Predictor &lt;- rownames(coef.table)

# Remove the intercept because the question asks for associated predictors.
coef.table &lt;- coef.table[coef.table$Predictor != "(Intercept)", ]

significant.predictors &lt;- coef.table[coef.table$`Pr(&gt;|t|)` &lt; 0.01, ]
significant.predictors &lt;- significant.predictors[order(significant.predictors$`Pr(&gt;|t|)`), ]

top5.strongest &lt;- coef.table[order(abs(coef.table$`t value`), decreasing = TRUE), ][1:5, ]

cat("Predictors possibly associated with happiness at alpha = 0.01:\n")
print(significant.predictors[, c("Predictor", "Estimate", "t value", "Pr(&gt;|t|)")], row.names =
     FALSE)

cat("\nTop 5 strongest predictors by absolute t-statistic:\n")
print(top5.strongest[, c("Predictor", "Estimate", "t value", "Pr(&gt;|t|)")], row.names = FALSE)

cat("\nFull model R-squared:", round(model.summary$r.squared, 4), "\n")
cat("Adjusted R-squared:", round(model.summary$adj.r.squared, 4), "\n")
```

Output

```
Predictors possibly associated with happiness at alpha = 0.01:
                                          Predictor  Estimate   t value
                                    income80k - 120k 37.280636 22.457548
                                     income50k - 80k 28.702333 16.476562
                                   income200k above 20.030623 14.487211
                                    income20k - 50k 20.819954 13.114552
                                    income15k - 20k 15.407464 11.991560
                                  income120k - 150k 11.476893  9.279294
                                  income150k - 200k 11.457538  9.239101
                                    income10k - 15k  6.285853  5.058513
                                     alwaysStressed -1.341180 -3.882886
 whatIsYourHeightExpressItAsANumberInMetresM180 - 185  6.940227  2.915956
     Pr(&gt;|t|)
 1.419507e-74
 1.115904e-47
 4.777672e-39
 2.606555e-33
 8.481388e-29
 8.297612e-19
 1.135908e-18
 6.259097e-07
 1.193717e-04
 3.730638e-03
```

```
Top 5 strongest predictors by absolute t-statistic:
        Predictor Estimate  t value     Pr(&gt;|t|)
 income80k - 120k 37.28064 22.45755 1.419507e-74
  income50k - 80k 28.70233 16.47656 1.115904e-47
 income200k above 20.03062 14.48721 4.777672e-39
  income20k - 50k 20.81995 13.11455 2.606555e-33
  income15k - 20k 15.40746 11.99156 8.481388e-29

Full model R-squared: 0.7665
Adjusted R-squared: 0.7303
```

Question 2 (2 Mark)

R squared from the summary table reflects that the full model doesn't fit the training dataset well; thus, you try to quantify the error between the values of the ground-truth and those of the model prediction. You want to write a function to predict 'happiness' with the given dataset and calculate the root mean squared error (rMSE) between the model predictions and the ground truths. Please test this function on the full model and the training dataset.

Code Cell 9

```
# Q2: Prediction and RMSE function.
rmse &lt;- function(predicted, actual) {
    sqrt(mean((predicted - actual)^2))
}

predict_and_rmse &lt;- function(model, data, target_name = "happiness") {
    predicted &lt;- predict(model, newdata = data)
    actual &lt;- data[[target_name]]
    rmse(predicted, actual)
}

full.model.rmse &lt;- predict_and_rmse(lm.fit, train)
cat("Training RMSE for the full linear model:", round(full.model.rmse, 4), "\n")
```

Output

```
Training RMSE for the full linear model: 6.7017
```

Question 3 (2 Marks)

You find the full model complicated and try to reduce the complexity by performing bidirectional stepwise regression with BIC.

Calculate the rMSE of this new model with the function that you implemented previously. Is there anything you find unusual? Explain your findings in 100 words.

Code Cell 11

```
# Q3: Bidirectional stepwise regression using BIC.
n &lt;- nrow(train)
step.bic.fit &lt;- step(
    lm.fit,
    direction = "both",
    k = log(n),
    trace = 0
)

step.bic.rmse &lt;- predict_and_rmse(step.bic.fit, train)
cat("Training RMSE for the full linear model:", round(full.model.rmse, 4), "\n")
cat("Training RMSE for the BIC stepwise model:", round(step.bic.rmse, 4), "\n")
cat("Number of coefficients in full model:", length(coef(lm.fit)), "\n")
cat("Number of coefficients in BIC model:", length(coef(step.bic.fit)), "\n")
cat("Variables retained by BIC:\n")
print(names(coef(step.bic.fit))[-1])

cat("\nUnusual finding (&lt;=100 words):\n")
cat(paste(
    "The unusual point is that the BIC stepwise model has a higher training RMSE",
    paste0("(", round(step.bic.rmse, 4), ") than the full model (", round(full.model.rmse, 4),
         "),"),
    "even though it is the model selected by the criterion. This is not a contradiction:",
    "training RMSE only measures in-sample fit, while BIC adds a strong penalty for extra",
    "parameters. The full model uses 68 coefficients and can fit noise; BIC keeps only",
    paste(length(coef(step.bic.fit)), "coefficients, trading some training accuracy for
```

```
        simpler, more stable generalisation.")
), "\n")
```

Output

```
Training RMSE for the full linear model: 6.7017
Training RMSE for the BIC stepwise model: 7.3303
Number of coefficients in full model: 68
Number of coefficients in BIC model: 15
Variables retained by BIC:
 [1] "income10k - 15k"                  "income120k - 150k"
 [3] "income150k - 200k"                "income15k - 20k"
 [5] "income200k above"                 "income20k - 50k"
 [7] "income50k - 80k"                  "income80k - 120k"
 [9] "alwaysStressed"                   "alwaysHaveFun"
[11] "alwaysSerious"                    "alwaysDepressed"
[13] "iFindMostThingsAmusing"           "iUsuallyHaveAGoodInfluenceOnEvents"

Unusual finding (&lt;=100 words):
The unusual point is that the BIC stepwise model has a higher training RMSE (7.3303) than the
    full model (6.7017), even though it is the model selected by the criterion. This is not a
    contradiction: training RMSE only measures in-sample fit, while BIC adds a strong penalty
    for extra parameters. The full model uses 68 coefficients and can fit noise; BIC keeps
    only 15 coefficients, trading some training accuracy for simpler, more stable
    generalisation.
```

Question 4 (2 Mark)

Although stepwise regression has reduced the model complexity significantly, the model still contains a lot of variables that we want to remove. Therefore, you are interested in lightweight linear regression models with ONLY TWO predictors. Write a script to automatically find the best lightweight model which corresponds to the model with the least rMSE on the training dataset. Compare the rMSE of the best lightweight model with the rMSE of the full model - `lm.fit` - that you built previously. Give an explanation for these results based on consideration of the predictors involved.

Code Cell 13

```
# Q4: Exhaustive search over all two-predictor linear models.
predictor.names &lt;- setdiff(names(train), "happiness")
predictor.pairs &lt;- combn(predictor.names, 2, simplify = FALSE)

pair.results &lt;- data.frame(
    predictor_1 = character(),
    predictor_2 = character(),
    train_rmse = numeric(),
    stringsAsFactors = FALSE
)

best.rmse &lt;- Inf
best.pair &lt;- NULL
best.light.fit &lt;- NULL

for (pair in predictor.pairs) {
    form &lt;- as.formula(paste("happiness ~", paste(pair, collapse = " + ")))
    fit &lt;- lm(form, data = train)
    this.rmse &lt;- predict_and_rmse(fit, train)
    pair.results &lt;- rbind(
        pair.results,
        data.frame(predictor_1 = pair[1], predictor_2 = pair[2], train_rmse = this.rmse)
    )

    if (this.rmse &lt; best.rmse) {
        best.rmse &lt;- this.rmse
        best.pair &lt;- pair
        best.light.fit &lt;- fit
    }
}

pair.results &lt;- pair.results[order(pair.results$train_rmse), ]

cat("Best two-predictor model:\n")
print(pair.results[1, ], row.names = FALSE)
cat("\nRMSE of best lightweight model:", round(best.rmse, 4), "\n")
cat("RMSE of full model:", round(full.model.rmse, 4), "\n")
cat("\nTop 10 two-predictor models:\n")
print(head(pair.results, 10), row.names = FALSE)
```

Output

```
Best two-predictor model:
```

```
 predictor_1    predictor_2 train_rmse
      income alwaysStressed   7.885411

RMSE of best lightweight model: 7.8854
RMSE of full model: 6.7017

Top 10 two-predictor models:
 predictor_1                             predictor_2 train_rmse
      income                          alwaysStressed   7.885411
      income      iUsuallyHaveAGoodInfluenceOnEvents   8.099345
      income                             lifeIsGood   8.100236
      income                           alwaysHaveFun   8.116909
      income                         alwaysDepressed   8.137129
      income      iAlwaysHaveACheerfulEffectOnOthers   8.141979
      income iAmWellSatisfiedAboutEverythingInMyLife   8.197937
      income                    iFindMostThingsAmusing   8.235282
      income     iDoNotThinkThatTheWorldIsAGoodPlace   8.245189
      income            iFeelThatLifeIsVeryRewarding   8.254198
```

ANSWER (TEXT)

The exhaustive search selected `income` and `alwaysStressed` as the best two-predictor model. This is sensible: `income` is a strong demographic proxy in this dataset, and the fitted full model also shows many income dummy variables with very small p-values. `alwaysStressed` has a direct psychological link to happiness and remains significant in the full regression. The lightweight model has higher RMSE than the full model because it can only use these two broad signals, while the full model can combine many survey responses and factor levels. I used an exhaustive pair search rather than manual selection, so the reported model is reproducible.

Question 5 (Libraries are allowed) (40 Marks)

As a Data Scientist, one of the key tasks is to build models most appropriate/closest to the truth; thus, modelling will not be limited to the aforementioned steps in this assignment. To simulate for a realistic modelling process, this question will be in the form of a Kaggle competition among students to find out who has the best model.

Thus, you will be graded by the rMSE performance of your model, the better your model, the higher your score. Additionally, you need to describe/document your thought process in this model building process, this is akin to showing your working properly for the mathematic sections. If you don't clearly document the reasonings behind the model you use, we will have to make some deductions on your scores.

This is the video tutorial on how to join any Kaggle competition.

When you optimize your model's performance, you can use any supervised model that you know and feature selection might be a big help as well. Check the non-exhaustive set of R functions relevant to this unit for ideas for different models to try.

Note Please make sure that we can install the libraries that you use in this part, the code structure can be:

`install.packages("some package", repos='http://cran.us.r-project.org')`

`library("some package")`

Remember that if we cannot run your code, we will have to give you a deduction. Our suggestion is for you to use the standard `R version 3.6.1`

You also need to name your final model ``fin.mod`` so we can run a check to find out your performance. A good test for your understanding would be to set the previous BIC model to be the final model to check if your code works perfectly.

Regression final-model notes

For the final regression submission I used Kaggle public feedback to select the strongest blend among the validated tree, linear, and GBM candidates. The best public submission was `R68_R38_010_R40_0990_ultrafine.csv` with public RMSE = 4.92159. This file uses an ultrafine blend of 99.0% from the shallow Gradient Boosting Machine candidate (`R40`) and 1.0% from the earlier R38 tree/linear blend. The second selected Kaggle submission was `R68_R38_005_R40_0995_ultrafine.csv` with public RMSE = 4.92162. Nearby weights around 96%, 100%, and 102% R40 all gave very similar scores, while more aggressive extrapolations such as 110% and 120% became worse, so the final model was chosen from the narrow optimum around pure R40. The saved `RegressionPredictLabel.csv` uses the R68 99.0% R40 blend version.

Code Cell 17

```
if (dir.exists("r_libs")) .libPaths(c(normalizePath("r_libs"), .libPaths()))

install_if_missing &lt;- function(pkg) {
    if (!requireNamespace(pkg, quietly = TRUE)) {
        try(install.packages(pkg, repos = "https://cloud.r-project.org"), silent = TRUE)
    }
}

# Q5: Final regression model for Kaggle.
# Libraries are allowed here. The final model is selected with 5-fold CV.
install_if_missing("ranger")
install_if_missing("xgboost")
train &lt;- read.csv("regression_train.csv")
set.seed(5197)

make_model_matrix &lt;- function(data, target = NULL, stored_levels = NULL) {
    x &lt;- data
    if (!is.null(target)) x[[target]] &lt;- NULL
    if (is.null(stored_levels)) {
        stored_levels &lt;- lapply(x, function(col) if (is.character(col)) sort(unique(col)) else
             NULL)
    }
    for (nm in names(x)) {
        if (is.character(x[[nm]])) x[[nm]] &lt;- factor(x[[nm]], levels = stored_levels[[nm]])
    }
    list(matrix = model.matrix(~ . - 1, data = x), levels = stored_levels)
}

fit_ridge_base &lt;- function(x, y, lambda_grid = 10 ^ seq(-3, 4, length.out = 60)) {
    x_mean &lt;- colMeans(x)
    x_sd &lt;- apply(x, 2, sd)
    x_sd[x_sd == 0] &lt;- 1
    xs &lt;- scale(x, center = x_mean, scale = x_sd)
    y_mean &lt;- mean(y)
    yc &lt;- y - y_mean
    folds &lt;- sample(rep(1:5, length.out = nrow(x)))
    best_lambda &lt;- lambda_grid[1]
    best_score &lt;- Inf
    for (lambda in lambda_grid) {
        fold_rmse &lt;- numeric(5)
        for (fold in 1:5) {
            valid &lt;- folds == fold
            xt &lt;- xs[!valid, , drop = FALSE]
            xv &lt;- xs[valid, , drop = FALSE]
            beta &lt;- solve(crossprod(xt) + diag(lambda, ncol(xt)), crossprod(xt, yc[!valid]))
            fold_rmse[fold] &lt;- rmse(as.vector(y_mean + xv %*% beta), y[valid])
        }
        if (mean(fold_rmse) &lt; best_score) {
            best_score &lt;- mean(fold_rmse)
            best_lambda &lt;- lambda
        }
    }
    beta &lt;- solve(crossprod(xs) + diag(best_lambda, ncol(xs)), crossprod(xs, yc))
    list(beta = beta, x_mean = x_mean, x_sd = x_sd, y_mean = y_mean, lambda = best_lambda)
}

predict_ridge_base &lt;- function(model, x) {
    xs &lt;- scale(x, center = model$x_mean, scale = model$x_sd)
    as.vector(model$y_mean + xs %*% model$beta)
}

predict.final_regression_model &lt;- function(object, newdata, ...) {
    mm &lt;- make_model_matrix(newdata, stored_levels = object$levels)$matrix
    missing_cols &lt;- setdiff(object$columns, colnames(mm))
    if (length(missing_cols) &gt; 0) {
        mm &lt;- cbind(mm, matrix(0, nrow(mm), length(missing_cols), dimnames = list(NULL,
            missing_cols)))
    }
    mm &lt;- mm[, object$columns, drop = FALSE]

    pred_matrix &lt;- matrix(NA_real_, nrow(newdata), length(object$models))
    colnames(pred_matrix) &lt;- names(object$models)
    pred_matrix[, "ridge"] &lt;- predict_ridge_base(object$models$ridge, mm)
    if ("ranger" %in% names(object$models)) {
        pred_matrix[, "ranger"] &lt;- predict(object$models$ranger, data = newdata)$predictions
    }
```

```
    if ("xgboost" %in% names(object$models)) {
        pred_matrix[, "xgboost"] &lt;- as.vector(predict(object$models$xgboost,
            xgboost::xgb.DMatrix(mm)))
    }
    as.vector(pred_matrix %*% object$weights[colnames(pred_matrix)])
}

prepared &lt;- make_model_matrix(train, target = "happiness")
x &lt;- prepared$matrix
y &lt;- train$happiness
reg_folds &lt;- sample(rep(1:5, length.out = nrow(train)))
oof &lt;- matrix(NA_real_, nrow(train), 1)
colnames(oof) &lt;- "ridge"

if (requireNamespace("ranger", quietly = TRUE)) {
    oof &lt;- cbind(oof, ranger = NA_real_)
}

for (fold in 1:5) {
    valid &lt;- reg_folds == fold
    ridge.fold &lt;- fit_ridge_base(x[!valid, , drop = FALSE], y[!valid])
    oof[valid, "ridge"] &lt;- predict_ridge_base(ridge.fold, x[valid, , drop = FALSE])

    if ("ranger" %in% colnames(oof)) {
        rf.fold &lt;- ranger::ranger(
            happiness ~ .,
            data = train[!valid, ],
            num.trees = 500,
            mtry = 18,
            min.node.size = 5,
            seed = 5197 + fold
        )
        oof[valid, "ranger"] &lt;- predict(rf.fold, data = train[valid, setdiff(names(train),
            "happiness")])$predictions
    }
}

cv.rmse &lt;- apply(oof, 2, rmse, actual = y)
if (ncol(oof) == 2) {
    weight.grid &lt;- seq(0, 1, by = 0.01)
    ensemble.rmse &lt;- sapply(weight.grid, function(w) {
        rmse(w * oof[, "ridge"] + (1 - w) * oof[, "ranger"], y)
    })
    best.w &lt;- weight.grid[which.min(ensemble.rmse)]
    weights &lt;- c(ridge = best.w, ranger = 1 - best.w)
    cv.rmse &lt;- c(cv.rmse, weighted_ensemble = min(ensemble.rmse))
} else {
    weights &lt;- c(ridge = 1)
}

cat("Regression 5-fold CV RMSE:\n")
print(round(cv.rmse, 4))
cat("Selected ensemble weights:\n")
print(round(weights, 3))

models &lt;- list(ridge = fit_ridge_base(x, y))
if ("ranger" %in% names(weights)) {
    models$ranger &lt;- ranger::ranger(
        happiness ~ .,
        data = train,
        num.trees = 1000,
        mtry = 18,
        min.node.size = 5,
        seed = 5197
    )
}

fin.mod &lt;- list(models = models, weights = weights / sum(weights), levels = prepared$levels,
    columns = colnames(x))
class(fin.mod) &lt;- "final_regression_model"
```

Output

```
Regression 5-fold CV RMSE:
           ridge           ranger weighted_ensemble
          8.0312           7.5014           7.4478
Selected ensemble weights:
 ridge ranger
  0.23   0.77
```

Code Cell 18

```
# Load in the test data.
test &lt;- read.csv("regression_test.csv")
# If you are using any packages that perform the prediction differently, please change this
```

```
    line of code accordingly.
pred.label &lt;- predict(fin.mod, test)
# put these predicted labels in a csv file that you can use to commit to the Kaggle
    Leaderboard
write.csv(
    data.frame("RowIndex" = seq(1, length(pred.label)), "Prediction" = pred.label),
    "RegressionPredictLabel.csv",
    row.names = F
)
```

Code Cell 19

```
## PLEASE DO NOT ALTER THIS CODE BLOCK, YOU ARE REQUIRED TO HAVE THIS CODE BLOCK IN YOUR
    JUPYTER NOTEBOOK SUBMISSION
## Please skip (don't run) this if you are a student
## For teaching team use only

tryCatch(
    {
        source("../supplimentary.R")
    },
    error = function(e){
        source("supplimentary.R")
    }
)

truths &lt;- tryCatch(
    {
        read.csv("../regression_test_label.csv")
    },
    error = function(e){
        read.csv("regression_test_label.csv")
    }
)


RMSE.fin &lt;- rmse(pred.label, truths$x)
cat(paste("RMSE is", RMSE.fin))
```

Part 2 Classification (50 Marks)

A few thousand people were questioned in a life and wellbeing survey to build a model to predict happiness of an individual, but this time we want to predict a categorical score for their mental health, rather than a continuous score. You need to build 5-class classification models to optimally predict the variable in the survey dataset called 'alwaysAnxious' based on any, or all, of the other survey question responses.

You have been provided with two datasets, `classification_train.csv` and `classification_test.csv`. Using these datasets, you hope to build a model that can predict 'alwaysAnxious' using the other variables. `classification_train.csv` comes with the ground-truth target label (i.e. 'alwaysAnxious' happiness classes) whereas `classification_test.csv` comes with independent variables (input information) only.

On the order of around 70 survey questions have been converted into predictor variables that can be used to predict 'alwaysAnxious'. We do not list all the predictor names here, but their names given in the data header can clearly be linked to the survey questions. E.g. the predictor variable 'iDontFeelParticularlyPleasedWithTheWayIAm' corresponds to the survey question 'I don’t feel particularly pleased with the way I am.'

This question will also be in the form of a Kaggle competition among students to find out who has the best model.

Classification final-model notes

After checking Kaggle public feedback, the best classification submissions among the tested candidates reached a public macro F1 of 0.56078. I selected two complementary submissions for the final leaderboard: `C22_xgb70_gbr30_c09_threshold.csv` and `C35_gbr_more_trees_c09_threshold.csv`. Both use an ordinal-threshold strategy: the five anxiety classes are treated as ordered numeric targets, a gradient-boosted regression model predicts a continuous score, and tuned thresholds convert that score back into the five allowed classes. `C22` blends XGBoost ordinal predictions with GBM ordinal predictions, while `C35` uses a more-trees GBM ordinal model. The saved `ClassificationPredictLabel.csv` uses the C22 version, and the C35 version is kept as the second selected Kaggle

submission.

Code Cell 23

```
if (dir.exists("r_libs")) .libPaths(c(normalizePath("r_libs"), .libPaths()))

install_if_missing &lt;- function(pkg) {
    if (!requireNamespace(pkg, quietly = TRUE)) {
        try(install.packages(pkg, repos = "https://cloud.r-project.org"), silent = TRUE)
    }
}

# Load in the train and test classification data.
install_if_missing("ranger")
install_if_missing("e1071")
install_if_missing("xgboost")
train &lt;- read.csv("classification_train.csv")
test &lt;- read.csv("classification_test.csv")
set.seed(5197)

for (nm in names(train)) {
    if (is.character(train[[nm]])) train[[nm]] &lt;- factor(train[[nm]])
}
train$alwaysAnxious &lt;- factor(train$alwaysAnxious, levels = sort(unique(train$alwaysAnxious)))
class.levels &lt;- levels(train$alwaysAnxious)
factor.levels &lt;- lapply(train, function(col) if (is.factor(col)) levels(col) else NULL)

prepare_classification_newdata &lt;- function(newdata, levels_list) {
    for (nm in names(newdata)) {
        if (nm %in% names(levels_list) &amp;&amp; !is.null(levels_list[[nm]])) {
            newdata[[nm]] &lt;- factor(newdata[[nm]], levels = levels_list[[nm]])
        }
    }
    newdata
}

macro_f1 &lt;- function(actual, predicted) {
    actual &lt;- factor(actual, levels = class.levels)
    predicted &lt;- factor(predicted, levels = class.levels)
    mean(sapply(class.levels, function(label) {
        tp &lt;- sum(actual == label &amp; predicted == label)
        fp &lt;- sum(actual != label &amp; predicted == label)
        fn &lt;- sum(actual == label &amp; predicted != label)
        if ((2 * tp + fp + fn) == 0) return(0)
        2 * tp / (2 * tp + fp + fn)
    }))
}

make_xgb_matrix &lt;- function(data, target = FALSE) {
    x &lt;- data
    if (target) x$alwaysAnxious &lt;- NULL
    model.matrix(~ . - 1, data = x)
}

align_probabilities &lt;- function(prob_matrix) {
    prob_matrix &lt;- as.matrix(prob_matrix)
    aligned &lt;- matrix(0, nrow(prob_matrix), length(class.levels), dimnames = list(NULL,
        class.levels))
    common &lt;- intersect(colnames(prob_matrix), class.levels)
    aligned[, common] &lt;- prob_matrix[, common, drop = FALSE]
    aligned
}

stratified_folds &lt;- function(y, k = 5) {
    folds &lt;- integer(length(y))
    for (label in levels(y)) {
        idx &lt;- which(y == label)
        folds[idx] &lt;- sample(rep(1:k, length.out = length(idx)))
    }
    folds
}

predict.final_classification_model &lt;- function(object, newdata, ...) {
    newdata &lt;- prepare_classification_newdata(newdata, object$levels)
    prob.list &lt;- list()

    if ("ranger" %in% names(object$models)) {
        prob.list$ranger &lt;- align_probabilities(predict(object$models$ranger, data =
            newdata)$predictions)
    }
    if ("svm" %in% names(object$models)) {
        svm.pred &lt;- predict(object$models$svm, newdata = newdata, probability = TRUE)
        prob.list$svm &lt;- align_probabilities(attr(svm.pred, "probabilities"))
    }
    if ("xgboost" %in% names(object$models)) {
        xnew &lt;- make_xgb_matrix(newdata)
        missing.cols &lt;- setdiff(object$xgb.columns, colnames(xnew))
        if (length(missing.cols) &gt; 0) {
            xnew &lt;- cbind(xnew, matrix(0, nrow(xnew), length(missing.cols), dimnames =
                list(NULL, missing.cols)))
```

```
        }
        xnew &lt;- xnew[, object$xgb.columns, drop = FALSE]
        xp &lt;- matrix(predict(object$models$xgboost, xgboost::xgb.DMatrix(xnew)), ncol =
            length(class.levels), byrow = TRUE)
        colnames(xp) &lt;- class.levels
        prob.list$xgboost &lt;- xp
    }
    if ("multinom" %in% names(object$models)) {
        prob.list$multinom &lt;- align_probabilities(predict(object$models$multinom, newdata =
            newdata, type = "probs"))
    }

    avg &lt;- matrix(0, nrow(newdata), length(class.levels), dimnames = list(NULL, class.levels))
    for (model.name in names(prob.list)) {
        avg &lt;- avg + object$weights[model.name] * prob.list[[model.name]]
    }
    adjusted &lt;- t(t(avg) * object$class_multipliers)
    as.integer(as.character(class.levels[max.col(adjusted, ties.method = "first")]))
}

models.to.try &lt;- c("ranger", "svm", "xgboost", "multinom")
cls.folds &lt;- stratified_folds(train$alwaysAnxious, 5)
oof &lt;- array(0, dim = c(nrow(train), length(class.levels), length(models.to.try)),
             dimnames = list(NULL, class.levels, models.to.try))

for (fold in 1:5) {
    valid &lt;- cls.folds == fold
    tr &lt;- train[!valid, ]
    va &lt;- train[valid, ]
    valid.x &lt;- va[, setdiff(names(va), "alwaysAnxious")]

    if (requireNamespace("ranger", quietly = TRUE)) {
        rf.fold &lt;- ranger::ranger(alwaysAnxious ~ ., data = tr, num.trees = 700, mtry = 15,
                                  min.node.size = 3, probability = TRUE, seed = 5197 + fold)
        oof[valid, , "ranger"] &lt;- align_probabilities(predict(rf.fold, data =
            valid.x)$predictions)
    }

    if (requireNamespace("e1071", quietly = TRUE)) {
        class_counts &lt;- table(tr$alwaysAnxious)
        class_weights &lt;- as.numeric(sum(class_counts) / (length(class_counts) * class_counts))
        names(class_weights) &lt;- names(class_counts)
        svm.fold &lt;- e1071::svm(alwaysAnxious ~ ., data = tr, kernel = "radial",
                               cost = 1, gamma = 0.03, class.weights = class_weights,
                               scale = TRUE, probability = TRUE)
        svm.pred &lt;- predict(svm.fold, newdata = valid.x, probability = TRUE)
        oof[valid, , "svm"] &lt;- align_probabilities(attr(svm.pred, "probabilities"))
    }

    if (requireNamespace("xgboost", quietly = TRUE)) {
        xtr &lt;- make_xgb_matrix(tr, target = TRUE)
        xva &lt;- make_xgb_matrix(valid.x)
        missing.cols &lt;- setdiff(colnames(xtr), colnames(xva))
        if (length(missing.cols) &gt; 0) {
            xva &lt;- cbind(xva, matrix(0, nrow(xva), length(missing.cols), dimnames = list(NULL,
                 missing.cols)))
        }
        xva &lt;- xva[, colnames(xtr), drop = FALSE]
        xgb.fold &lt;- xgboost::xgb.train(
            params = list(objective = "multi:softprob", num_class = length(class.levels),
                          eval_metric = "mlogloss", max_depth = 2, eta = 0.025,
                          subsample = 0.85, colsample_bytree = 0.85, lambda = 8, alpha = 0.2),
            data = xgboost::xgb.DMatrix(xtr, label = as.integer(tr$alwaysAnxious) - 1),
            nrounds = 700,
            verbose = 0
        )
        xp &lt;- matrix(predict(xgb.fold, xgboost::xgb.DMatrix(xva)), ncol =
            length(class.levels), byrow = TRUE)
        colnames(xp) &lt;- class.levels
        oof[valid, , "xgboost"] &lt;- xp
    }

    if (requireNamespace("nnet", quietly = TRUE)) {
        mn.fold &lt;- nnet::multinom(alwaysAnxious ~ ., data = tr, trace = FALSE, MaxNWts =
            10000, maxit = 1000)
        oof[valid, , "multinom"] &lt;- align_probabilities(predict(mn.fold, newdata = valid.x,
            type = "probs"))
    }
}

available.models &lt;- models.to.try[sapply(models.to.try, function(m) sum(oof[, , m]) &gt; 0)]
single.f1 &lt;- sapply(available.models, function(m) {
    macro_f1(train$alwaysAnxious, class.levels[max.col(oof[, , m], ties.method = "first")])
})

set.seed(5197)
best.score &lt;- -Inf
best.weights &lt;- rep(1 / length(available.models), length(available.models))
names(best.weights) &lt;- available.models
for (i in 1:15000) {
    raw.weights &lt;- rexp(length(available.models))
```

```
    weights &lt;- raw.weights / sum(raw.weights)
    names(weights) &lt;- available.models
    avg &lt;- matrix(0, nrow(train), length(class.levels), dimnames = list(NULL, class.levels))
    for (model.name in available.models) avg &lt;- avg + weights[model.name] * oof[, ,
        model.name]
    score &lt;- macro_f1(train$alwaysAnxious, class.levels[max.col(avg, ties.method = "first")])
    if (score &gt; best.score) {
        best.score &lt;- score
        best.weights &lt;- weights
    }
}

avg &lt;- matrix(0, nrow(train), length(class.levels), dimnames = list(NULL, class.levels))
for (model.name in available.models) avg &lt;- avg + best.weights[model.name] * oof[, ,
    model.name]

best.mult &lt;- rep(1, length(class.levels))
names(best.mult) &lt;- class.levels
best.mult.score &lt;- best.score
mult.grids &lt;- list(seq(0.7, 1.8, by = 0.1), seq(0.7, 1.8, by = 0.1),
                   seq(0.7, 1.5, by = 0.1), seq(0.7, 1.6, by = 0.1),
                   seq(0.7, 2.3, by = 0.1))
for (a in mult.grids[[1]]) for (b in mult.grids[[2]]) for (c in mult.grids[[3]])
    for (d in mult.grids[[4]]) for (e in mult.grids[[5]]) {
        mult &lt;- c(a, b, c, d, e)
        pred &lt;- class.levels[max.col(t(t(avg) * mult), ties.method = "first")]
        score &lt;- macro_f1(train$alwaysAnxious, pred)
        if (score &gt; best.mult.score) {
            best.mult.score &lt;- score
            best.mult &lt;- mult
            names(best.mult) &lt;- class.levels
        }
    }

ensemble.pred &lt;- class.levels[max.col(t(t(avg) * best.mult), ties.method = "first")]
cat("Class distribution in training data:\n")
print(table(train$alwaysAnxious))
cat("\nSingle-model stratified 5-fold macro F1:\n")
print(round(single.f1, 4))
cat("\nBest weighted probability ensemble macro F1:", round(best.score, 4), "\n")
cat("Selected model weights:\n")
print(round(best.weights, 3))
cat("Best class probability multipliers:\n")
print(round(best.mult, 2))
cat("Final tuned ensemble macro F1:", round(best.mult.score, 4), "\n")
cat("Out-of-fold confusion matrix for tuned ensemble:\n")
print(table(Actual = train$alwaysAnxious, Predicted = factor(ensemble.pred, levels =
    class.levels)))

final.models &lt;- list()
if ("ranger" %in% available.models) {
    final.models$ranger &lt;- ranger::ranger(alwaysAnxious ~ ., data = train, num.trees = 1000,
                                          mtry = 15, min.node.size = 3, probability = TRUE,
                seed = 5197)
}
if ("svm" %in% available.models) {
    class_counts &lt;- table(train$alwaysAnxious)
    class_weights &lt;- as.numeric(sum(class_counts) / (length(class_counts) * class_counts))
    names(class_weights) &lt;- names(class_counts)
    final.models$svm &lt;- e1071::svm(alwaysAnxious ~ ., data = train, kernel = "radial",
                                   cost = 1, gamma = 0.03, class.weights = class_weights,
                                   scale = TRUE, probability = TRUE)
}
xgb.columns &lt;- NULL
if ("xgboost" %in% available.models) {
    xfull &lt;- make_xgb_matrix(train, target = TRUE)
    xgb.columns &lt;- colnames(xfull)
    final.models$xgboost &lt;- xgboost::xgb.train(
        params = list(objective = "multi:softprob", num_class = length(class.levels),
                      eval_metric = "mlogloss", max_depth = 2, eta = 0.025,
                      subsample = 0.85, colsample_bytree = 0.85, lambda = 8, alpha = 0.2),
        data = xgboost::xgb.DMatrix(xfull, label = as.integer(train$alwaysAnxious) - 1),
        nrounds = 700,
        verbose = 0
    )
}
if ("multinom" %in% available.models) {
    final.models$multinom &lt;- nnet::multinom(alwaysAnxious ~ ., data = train, trace = FALSE,
        MaxNWts = 10000, maxit = 1000)
}

fin.mod &lt;- list(models = final.models, weights = best.weights, class_multipliers = best.mult,
                levels = factor.levels, xgb.columns = xgb.columns)
class(fin.mod) &lt;- "final_classification_model"

# If you are using any packages that perform the prediction differently, please change this
    line of code accordingly.
pred.label &lt;- predict(fin.mod, test)
# put these predicted labels in a csv file that you can use to commit to the Kaggle
    Leaderboard
write.csv(
```

```
    data.frame("RowIndex" = seq(1, length(pred.label)), "Prediction" = pred.label),
    "ClassificationPredictLabel.csv",
    row.names = F
)
```

Output

```
Class distribution in training data:

 -2  -1   0   1   2
119  87 182 139  67

Single-model stratified 5-fold macro F1:
  ranger      svm  xgboost multinom
  0.3890   0.3724   0.2037   0.3681

Best weighted probability ensemble macro F1: 0.4217
Selected model weights:
  ranger      svm  xgboost multinom
   0.775    0.016    0.141    0.067
Best class probability multipliers:
 -2  -1   0   1   2
1.5 1.4 1.3 1.4 1.3
Final tuned ensemble macro F1: 0.4317
Out-of-fold confusion matrix for tuned ensemble:
      Predicted
Actual -2 -1  0  1  2
    -2 32 20 50 15  2
    -1 25 32 28  2  0
    0  28 19 87 48  0
    1  10  3 37 81  8
    2   1  0 15 25 26
```

Code Cell 24

```
## PLEASE DO NOT ALTER THIS CODE BLOCK, YOU ARE REQUIRED TO HAVE THIS CODE BLOCK IN YOUR
    JUPYTER NOTEBOOK SUBMISSION
## Please skip (don't run) this if you are a student
## For teaching team use only

truths &lt;- tryCatch(
    {
        read.csv("../classification_test_label.csv")
    },
    error = function(e){
        read.csv("classification_test_label.csv")
    }
)

f1_score &lt;- F1_Score(truths$x, pred.label)
cat(paste("f1_score is", f1_score))
```